IEEE CONFERENCE PUBLICATION ANALYSIS

Comparative Research Report: Base Papers vs. Grounded Multi-Agent RAV Platform

Final Year Project Benchmark Analysis & Target Performance Specifications

Executive Summary

This report presents an in-depth comparative evaluation between four state-of-the-art base papers in automated software engineering and our proposed system: **A Grounded Multi-Agent Framework for Autonomous Legacy Software Intelligence (RAV Pipeline)**.

Our system replaces conventional heuristic Information Retrieval (IR) and standard unverified Retrieval-Augmented Generation (RAG) with a deterministic, multi-relational **Semantic Software Knowledge Graph (SSKG)** coupled with a **Deterministic Verification Gate**. The benchmark target specifications demonstrate superior targeted precision (**76.5% – 79.2%**) and a reduced hallucination ceiling (**7.5% – 9.8%**) compared to traditional models, while maintaining strict, verifiable code-to-line traceability across **13+ programming languages**.

1. Complete Analysis of Base Papers

Paper 1: Recovering Traceability Links Between Code and Documentation: A Retrospective

IEEE Transactions on Software Engineering (TSE 2025 Retrospective on 2002 Seminal Work)
Authors: Giulio Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, Ettore Merlo

- **Core Focus & Objective:** Automatically recovering traceability links between source code entities (functions, classes, variables) and natural-language legacy documentation (requirements, design docs, user manuals).
- **Algorithms & Methodologies Used:**
  1. **Vector Space Model (VSM):** Represents code and documents as term vectors, using **TF-IDF** (Term Frequency-Inverse Document Frequency) weighting and **Cosine Similarity** to compute link likelihood.
  2. **Probabilistic IR Models:** Estimates the probability that a document generates specific code terms.
  3. **Lexical Preprocessing:** Identifier splitting (camelCase/snake_case), stop-word removal, and Porter Stemming.
  4. **Topic Modeling Extensions:** Latent Semantic Indexing (LSI) and Latent Dirichlet Allocation (LDA) to group synonyms into abstract concept spaces.
- **Evaluation Criteria & Metrics:**
  - **Precision:** $\frac{|\text{Correct Links Retrieved}|}{|\text{Total Links Retrieved}|}$
  - **Recall:** $\frac{|\text{Correct Links Retrieved}|}{|\text{Total Links in Ground Truth}|}$
  - **Cut-point & Similarity Thresholds:** Evaluates precision at fixed rank cut-offs.
- **Core Limitations:** Severe vocabulary mismatch between developer code identifiers and formal natural-language prose; high false-positive rates (40); lack of semantic code understanding; zero automated verification.

Paper 2: IntelliUnitGen: A Unit Test Case Generation Framework Based on Integration of Static Analysis and Prompt Learning

IEEE Access 2025
Authors: Zhuoying Yang, Lei Wang

- **Core Focus & Objective:** Generating executable, high-coverage JUnit 5 unit test cases for complex Java applications by integrating JavaParser static analysis with LLMs.
- **Algorithms & Methodologies Used:**
  1. **JavaParser Static Code Analysis:** Extracts Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), Data Flow DUPs (Def-Use Pairs), dependency chains, and cyclomatic complexity.
  2. **Chain-of-Thought (CoT) Prompt Synthesis:** Multi-turn hierarchical prompt templates decomposing test generation into structure understanding $\rightarrow$ execution condition analysis $\rightarrow$ test generation.
  3. **In-Context Few-Shot Learning:** Supplies exemplar test patterns matching target frameworks (JUnit 5, PyTest).
  4. **Automated Reflection & Error Repair:** Runs Maven compilation commands (`mvn test`) and feeds back error logs to iteratively re-prompt the LLM (up to 15 reflection rounds).
- **Evaluation Criteria & Metrics:**
  - **Statement Coverage & Branch Coverage (%)** (measured via JaCoCo)
  - **First-Pass Compilation Success Rate (FPCSR):** $\frac{\text{Compilable Test Classes}}{\text{Total Generated Test Classes}} \times 100$
  - **First-Pass Execution Success Rate (FPESR):** $\frac{\text{Runnable Test Classes}}{\text{Total Generated Test Classes}} \times 100$
- **Core Limitations:** Limited to unit test generation for isolated classes/methods; single-language focus (Java); depends on closed-source cloud APIs (GPT-4o); lacks system-wide architectural knowledge representation.

Paper 3: Evaluating Retrieval-Augmented Generation for LLM-Based Vulnerability Detection: An Empirical Study on Real-World Java Vulnerabilities

IEEE Access 2026
Authors: Gábor Antal, Lóránt Értekes, Norbert Szolnoki, Péter Hegedűs

- **Core Focus & Objective:** Benchmarking whether standard RAG improves the capability of 7 SOTA LLMs (GPT-5, GPT-4o, Claude Sonnet 4, Gemini 2.5, DeepSeek 3.1, Grok Code Fast, LLaMA 4) to classify software vulnerabilities in real-world Java projects.
- **Algorithms & Methodologies Used:**
  1. **Dense Vector Embeddings & Similarity Search:** Uses OpenAI `text-embedding-3-small` in **ChromaDB** vector database with Cosine Similarity thresholding (≥ 0.6).
  2. **Longest Common Subsequence (LCS) Syntactic Overlap Filter:** Normalizes code string similarity to prevent data leakage between vector database and evaluation set:
  $$\text{sim}(c_1, c_2) = \frac{1}{2}\left(\frac{|\text{LCS}(c_1, c_2)|}{|c_1|} + \frac{|\text{LCS}(c_1, c_2)|}{|c_2|}\right)$$
  3. **Majority Voting Ensemble:** Combines outputs of 3 to 5 LLMs to improve overall accuracy.
- **Evaluation Criteria & Metrics:**
  - **False Positive Rate (FPR)** and **False Negative Rate (FNR)**
  - **Accuracy, Precision, Recall, F1-Score**
  - **Matthews Correlation Coefficient (MCC):** Measures binary classification quality on imbalanced sets.
  - **Pairwise Discrimination Accuracy:** Measures ability to distinguish vulnerable code from its exact patched version.

- **Core Limitations:** High persistent false-positive rates (54.63 average FPR); dense vector retrieval failed to return context in **61% of queries** (causing fallback to zero-shot); misclassifies fixed code as vulnerable.

Paper 4: A Survey on Application of AI on Reverse Engineering for Software Analysis and Security

IEEE Access 2025
Authors: Ashutosh Ghimire, Sahasra Rao Lingala, Junjie Zhang, Faris Alsulami, Fathi Amsaad

- **Core Focus & Objective:** Surveying AI and Machine Learning techniques in software reverse engineering, decompilation, binary analysis, malware classification, and obfuscation detection.
- **Algorithms & Methodologies Surveyed:**
  1. **Binary Static/Dynamic Tools:** Ghidra, IDA Pro, Radare2 disassemblers, LLVM IR, VEX IR.
  2. **Traditional ML Classifiers:** Decision Trees, Support Vector Machines (SVM), Random Forests for binary feature classification.
  3. **Deep Learning Architectures:** Convolutional Neural Networks (CNNs) on raw binary byte-images (MalNet), LSTMs, and Graph Neural Networks (GNNs like Devign) operating on Control Flow Graphs (CFGs).
  4. **Transformer Decompilers:** CodeBERT, CodeT5, LLM4Decompile for assembly-to-source translation.
- **Evaluation Criteria & Metrics:**
  - **Classification Accuracy, F1-Score, Execution Speed**
  - **Decompilation BLEU Score**
  - **Obfuscation Defeat Rate**
- **Core Limitations:** Identifies major industry gaps: black-box deep learning models lack explainability; static reverse engineering tools fail against complex obfuscation; existing models focus on binary decompilation rather than high-level codebase understanding and architectural documentation.

---

2. Algorithms & Methodologies Used in OUR Project

Our project, **A Grounded Multi-Agent Framework for Autonomous Legacy Software Intelligence (RAV Pipeline)**, uses a multi-layered hybrid architecture:

Component	Algorithm / Method	Description / Formula
Agent 4 (Parser Agent)	Tree-sitter Grammar Router	Universal, multi-language AST extraction of classes, methods, fields, annotations, imports, and API routes across 13+ languages.
Agent 5 & `graph/sskg.py`	NetworkX Directed Multi-Graph	Constructs a **Semantic Software Knowledge Graph (SSKG)** with 9 node types (Class, Method, API, Entity...) and 8 relationship types (CALLS, EXTENDS, IMPLEMENTS...).
Agent 7 (Retrieval Agent)	Topology-Aware Subgraph Retrieval	Traverses SSKG graph topology to retrieve explicit multi-relational code context matching queries (replacing vector similarity search).
Agent 8 (Reconstruction)	Grounded Context Prompting	Enforces strict context-only generation prompts using local Ollama (Llama 3).
Agent 9 (Verification)	Deterministic Grounded RAV Verification Gate	Computes non-LLM confidence scores and evidence citations: $\text{Confidence} = \alpha \cdot \text{EvidenceMatch} + \beta \cdot \text{GraphConsistency} + \gamma \cdot \text{RetrievalCoverage}$

Component	Algorithm / Method	Description / Formula
Agent 6 (Diagram Agent)	AST-to-Mermaid/PlantUML Engine	Deterministically renders UML Class, Sequence, Component, and ER diagrams directly from AST data without LLM involvement.
Agent 11 (Evaluation)	Dynamic Real-Time Metric Engine	Calculates Precision, Recall, F1, Hallucination Rate, and Citation Accuracy from live AST counts and verification evidence.

3. Comprehensive Comparison Table

Metric / Feature	Paper 1 (Antoniol 2025)	Paper 2 (Yang 2025 - IntelliUnitGen)	Paper 3 (Antal 2026 - AVD RAG)	Paper 4 (Ghimire 2025 - Survey)	OUR PROJECT (Grounded RAV)
Primary Focus	Traceability Link Recovery	Unit Test Generation	Vulnerability Detection	Reverse Engineering Survey	**Complete Legacy Codebase Intelligence & Documentation**
Core Algorithm	Vector Space Model (VSM), TF-IDF, Cosine Similarity	JavaParser AST + CoT Prompting + Reflection	ChromaDB Dense Embeddings + Cosine Similarity	CNN, GNN, Transformer Decompilers	**Tree-sitter AST + NetworkX SSKG + RAV Pipeline**
Grounding Mechanism	Text Similarity matching	Static Code Prompts	Vector Database Retrieval	None / Model weights	**Multi-relational Graph Subgraph Retrieval**
Verification Method	Manual Developer Review	Maven Build Compilation & Test Execution	Manual Pairwise Check	N/A (Survey)	**Deterministic Algorithmic Verification (Confidence Score)**
Evidence Citation	None (Ranked list of docs)	None	Retrieved Code Snippets	N/A	**Exact File Path + Line Number Range Citations**
Hallucination Control	High False Positives	LLM Error-Retry (up to 15 runs)	High False Positives (>50%)	High in Deep Learning models	**Deterministic Verification Gate (< 9.8% Target Ceiling)**
Language Support	Language Agnostic (Text-level)	Java Only	Java Only	Multi-language (Binary level)	**13+ Languages (Java, Python, JS/TS, Go, C#, Ruby, Rust, etc.)**
Output Produced	Ranked link lists	JUnit 5 Test Files	Binary Vulnerable/Secure Label	Binary / Malware labels	**Interactive React Flow Graphs, UML Diagrams, Verified Docs, Q&A**

4. Benchmark Accuracy Analysis & Target Performance Metrics

> 💡 **IMPORTANT**
>
> **Methodological Note on Metrics:** The figures presented below for Base Papers (Papers 1–4) represent empirical results published in their respective peer-reviewed literature. For **Our System (Grounded RAV)**, the listed performance figures reflect the **designed target performance specifications and evaluation benchmark thresholds** established for empirical validation in upcoming ablation runs.

Performance Reached by Base Papers vs. Target System Specifications

- **Paper 1 (VSM / TF-IDF):** Achieves **40% – 60% Precision** due to vocabulary mismatch between source code variable names and natural language prose.
- **Paper 2 (IntelliUnitGen):** Achieves **67.0% Statement Coverage** and **60.7% First-Pass Compilation Success Rate (FPCSR)**.
- **Paper 3 (RAG Vulnerability Detection):** Achieves **53.0% – 62.96% Accuracy**, **54.6% – 67.0% F1-Score**, but suffers from **54.63% False Positive Rate** and **61% retrieval failure rate**.
- **Paper 4 (Deep Learning / GNNs):** Deep Learning models achieve **25% – 44% F1-Score** and suffer significant performance drops on unseen codebases.
- **OUR PROJECT (Target Benchmark Parameters for Grounded RAV Pipeline):**
  - **Target Precision Range: 76.5% – 79.2%** (designed target specification exceeding base papers' 63.4% – 67.0% baseline).
  - **Target Recall & F1-Score: 76.0% – 78.8%** (designed target specification exceeding base papers' 54.6% – 67.0% baseline).
  - **Target Hallucination Ceiling: 7.5% – 9.8%** (compared to 40% – 62.4% false-positive/unsupported rates in traditional RAG baselines).
  - **Target Citation Accuracy: 78.0% – 82.5%** with strict, deterministic 100% verifiable file path and line number citations.

---

5. Key Improvements over Base Papers

1. **Deterministic Verification Gate (RAV vs. Standard RAG):** Unlike Paper 3, which feeds vector-retrieved text directly into the LLM without validation, our **Agent 9 (Verification Agent)** evaluates every AI claim using an algorithmic formula ($\alpha\text{EvidenceMatch} + \beta\text{GraphConsistency} + \gamma\text{RetrievalCoverage}$). Claims falling below the designed minimum confidence threshold of **0.82** are flagged as `UNSUPPORTED`.
2. **Topological Knowledge Graph (SSKG) vs. Flat Vector Embeddings:** Standard vector databases (ChromaDB in Paper 3) lose structural relationships. Our **NetworkX SSKG** preserves multi-relational topology (`Class` → `Method` → `Repository` → `DatabaseTable`), aiming to eliminate the 61% retrieval failure observed in Paper 3.
3. **Multi-Language Tree-sitter Parser Router:** While Paper 2 and Paper 3 are restricted to Java, our system supports **13+ programming languages** (Java, Python, JS/TS, Go, C#, Ruby, PHP, Rust, Kotlin, Swift, Scala, C/C++) with dynamic framework detection.
4. **Hallucination-Free UML Diagram Generation:** Instead of using LLMs to draw diagrams (which causes hallucinated class names), **Agent 6 (Diagram Agent)** extracts structural relationships straight from AST data to generate exact Mermaid.js, PlantUML, and SVG diagrams.

---

6. Remaining Limitations Being Addressed

1. **Complex Inter-Class Dependency Depth in Highly Dynamic Languages:**
   - *Current Limitation:* JavaScript/Python code using duck typing or highly dynamic reflection can occasionally produce incomplete edge connections in the SSKG.
   - *Mitigation in Progress:* Enhancing Agent 4 with dynamic runtime tracing and fallback reflection analysis.
2. **Large File Context Window Constraints:**
   - *Current Limitation:* Extremely monolithic source files (> 5,000 lines) can exceed local LLM prompt windows during conversational Q&A.
   - *Mitigation in Progress:* Implementing AST-based method slicing before context injection in Agent 10.
3. **Local LLM Inference Latency:**
   - *Current Limitation:* Running 11 sequential agents on local Ollama Llama 3 requires moderate execution time for very large codebases.
   - *Mitigation in Progress:* Introducing parallel multi-agent execution branches for non-dependent pipeline stages (e.g., running Agent 3A and Agent 4 concurrently).